

```

import re
import subprocess
import os
from datetime import datetime

# Configuration variables
LOG_FILE = "/var/log/secure"
BANNED_IPS_FILE = "/var/log/banned_ips_audit.log"
FAILED_THRESHOLD = 5

def get_already_banned_ips():
    """Reads the audit log to build a set of IPs that are already blocked."""
    banned_ips = set()
    if os.path.exists(BANNED_IPS_FILE):
        with open(BANNED_IPS_FILE, "r") as file:
            for line in file:
                # Extracts the IP from the formatted log line
                parts = line.strip().split(" - BANNED - ")
                if len(parts) == 2:
                    banned_ips.add(parts[1])
    return banned_ips

def analyze_logs(banned_ips):
    failed_attempts = {}

    # Regex to match the standard RHEL failed SSH login log format
    ip_pattern = re.compile(r"Failed password for .* from ([0-9]+\.[0-9]+\.[0-9]+\.[0-9]+)")

```

```
if not os.path.exists(LOG_FILE):  
    print(f"Error: {LOG_FILE} not found. Ensure this is running on a RHEL/CentOS system.")  
    return failed_attempts
```

```
# Read the log file and extract IPs
```

```
with open(LOG_FILE, "r") as file:
```

```
    for line in file:
```

```
        match = ip_pattern.search(line)
```

```
        if match:
```

```
            ip = match.group(1)
```

```
            # Only count the IP if it hasn't been banned yet
```

```
            if ip not in banned_ips:
```

```
                failed_attempts[ip] = failed_attempts.get(ip, 0) + 1
```

```
    return failed_attempts
```

```
def block_ip(ip):
```

```
    try:
```

```
        print(f"[{datetime.now()}] Action Required: Blocking Malicious IP {ip}...")
```

```
        # Interface directly with firewalld to drop traffic from the IP
```

```
        rule = f"rule family='ipv4' source address='{ip}' drop"
```

```
        # Added text=True and capture_output to handle errors gracefully
```

```
        subprocess.run(
```

```
["firewall-cmd", "--permanent", "--add-rich-rule", rule],  
check=True,  
stdout=subprocess.DEVNULL,  
stderr=subprocess.PIPE,  
text=True  
)
```

```
subprocess.run(  
    ["firewall-cmd", "--reload"],  
    check=True,  
    stdout=subprocess.DEVNULL,  
    stderr=subprocess.PIPE  
)
```

```
# Log the action for auditing purposes
```

```
with open(BANNED_IPS_FILE, "a") as audit_log:
```

```
    audit_log.write(f"{datetime.now()} - BANNED - {ip}\n")
```

```
print(f"Success: {ip} has been permanently blocked.")
```

```
except subprocess.CalledProcessError as e:
```

```
    # Extracts the specific error message from firewalld instead of a raw Python traceback
```

```
    error_msg = e.stderr.strip() if e.stderr else str(e)
```

```
    print(f"System Error: Failed to apply firewall rule for {ip}. Details: {error_msg}")
```

```
def main():
```

```
    print(f"[{datetime.now()}] Starting RHEL Security Audit...")
```

```
# Load previously banned IPs into memory
banned_ips = get_already_banned_ips()
attempts = analyze_logs(banned_ips)
action_taken = False

for ip, count in attempts.items():
    if count >= FAILED_THRESHOLD:
        print(f"Warning: IP {ip} detected with {count} failed login attempts.")
        block_ip(ip)
        action_taken = True

if not action_taken:
    print("Audit Complete: No malicious thresholds exceeded.")

if __name__ == "__main__":
    # Note: Script must be run as root to access /var/log/secure and firewallld
    main()
```